

1. Demonstrate 7 Exceptions

```
import java.util.Scanner;
import java.util.InputMismatchException;

public class ExceptionDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }

        try {
            int a[] = {10, 20, 30};
            System.out.println(a[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index.");
        }

        try {
            System.out.print("Enter your roll number: ");
            String s = sc.next();
            int roll = Integer.parseInt(s);
        } catch (NumberFormatException e) {
            System.out.println("Marks must be numeric.");
        }

        try {
            System.out.print("Enter an integer: ");
            int n = sc.nextInt();
        } catch (InputMismatchException e) {
            System.out.println("Please enter an integer.");
            sc.nextLine();
        }

        try {
            String name = null;
            System.out.println(name.length());
        } catch (NullPointerException e) {
            System.out.println("Student name is null.");
        }

        try {
            String name = "Java";
            System.out.println(name.charAt(10));
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("Invalid character index.");
        }

        try {
            int marks = 120;

            if (marks < 0 || marks > 100) {
```

```

        throw new IllegalArgumentException();
    }
} catch (IllegalArgumentException e) {
    System.out.println("Marks must be between 0 and 100.");
}
}
}

```

2. Student Marks System – Multiple Catch

```

import java.util.Scanner;
import java.util.InputMismatchException;

public class StudentMarks {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            int marks[] = new int[3];

            for (int i = 0; i < 3; i++) {
                System.out.print("Enter marks: ");
                marks[i] = sc.nextInt();
            }

            int subjects = 3;

            if (subjects == 0) {
                throw new ArithmeticException();
            }

            int average = (marks[0] + marks[1] + marks[2]) / subjects;

            System.out.println("Average = " + average);

            System.out.println(marks[5]);

            String s = "abc";
            int n = Integer.parseInt(s);

        } catch (InputMismatchException e) {
            System.out.println("Please enter numeric marks.");
        } catch (ArithmeticException e) {
            System.out.println("Cannot calculate average.");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid marks array index.");
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format.");
        }
    }
}

```

3. Student Result System – Nested Try-Catch

```

import java.util.Scanner;
import java.util.InputMismatchException;

public class StudentResult {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter student name: ");
            String name = sc.nextLine();

            int marks[] = new int[3];

            for (int i = 0; i < 3; i++) {
                System.out.print("Enter marks: ");
                marks[i] = sc.nextInt();
            }

            try {
                int subjects = 3;

                try {
                    System.out.println("Subject mark: " + marks[5]);
                } catch (ArrayIndexOutOfBoundsException e) {
                    System.out.println("Invalid subject index.");
                }

                if (subjects == 0) {
                    throw new ArithmeticException();
                }

                int total = marks[0] + marks[1] + marks[2];
                double average = total / subjects;

                System.out.println("Student Name: " + name);
                System.out.println("Average: " + average);

            } catch (ArithmeticException e) {
                System.out.println("Cannot calculate average.");
            }

        } catch (InputMismatchException e) {
            System.out.println("Please enter marks as numbers.");
        }
    }
}

```

4. Bank Account – Insufficient Balance

```

import java.util.Scanner;

public class BankWithdrawal {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}

```

```

double balance = 10000;

System.out.print("Enter account holder name: ");
String name = sc.nextLine();

System.out.print("Enter withdrawal amount: ");
double amount = sc.nextDouble();

try {
    if (amount > balance) {
        throw new IllegalArgumentException("Insufficient balance");
    }

    balance = balance - amount;

    System.out.println("Withdrawal successful");
    System.out.println("Remaining Balance: ₹" + balance);

} catch (IllegalArgumentException e) {
    System.out.println(e.getMessage());
}
}
}

```

5. ATM Transaction Using Two Threads

```

class ATMTransaction implements Runnable {
    public void run() {
        try {
            System.out.println("Card Inserted");
            Thread.sleep(1000);

            System.out.println("PIN Verified");
            Thread.sleep(1000);

            System.out.println("Transaction Processing");
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

class AccountNotification implements Runnable {
    public void run() {
        try {
            System.out.println("Checking Balance");
            Thread.sleep(1000);

            System.out.println("Amount Debited");
            Thread.sleep(1000);

            System.out.println("SMS Sent");
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

```

```

    }
}

public class ATM {
    public static void main(String[] args) {
        ATMTransaction t1 = new ATMTransaction();
        AccountNotification t2 = new AccountNotification();

        Thread thread1 = new Thread(t1);
        Thread thread2 = new Thread(t2);

        thread1.start();
        thread2.start();
    }
}

```

6. Bank Account Using Threads

```

class Account {
    double balance = 5000;

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Insufficient balance");
        }
    }
}

class Deposit extends Thread {
    Account account;

    Deposit(Account account) {
        this.account = account;
    }

    public void run() {
        account.deposit(2000);
    }
}

class Withdraw extends Thread {
    Account account;

    Withdraw(Account account) {
        this.account = account;
    }
}

```

```

    }

    public void run() {
        account.withdraw(1000);
    }
}

public class BankThreads {
    public static void main(String[] args) {
        Account account = new Account();

        Deposit d = new Deposit(account);
        Withdraw w = new Withdraw(account);

        d.start();
        w.start();
    }
}

```

7. Thread Priority – Deposit and Withdraw

```

class Account {
    double balance = 5000;

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Deposit: " + amount);
        System.out.println("Balance: " + balance);
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdraw: " + amount);
            System.out.println("Balance: " + balance);
        }
    }
}

class Deposit extends Thread {
    Account account;

    Deposit(Account account) {
        this.account = account;
    }

    public void run() {
        account.deposit(2000);
    }
}

class Withdraw extends Thread {
    Account account;

    Withdraw(Account account) {

```

```

        this.account = account;
    }

    public void run() {
        account.withdraw(1000);
    }
}

public class ThreadPriority {
    public static void main(String[] args) {
        Account account = new Account();

        Deposit d = new Deposit(account);
        Withdraw w = new Withdraw(account);

        d.setPriority(Thread.MAX_PRIORITY);
        w.setPriority(Thread.MIN_PRIORITY);

        System.out.println("Deposit Priority: " + d.getPriority());
        System.out.println("Withdraw Priority: " + w.getPriority());

        d.start();
        w.start();
    }
}

```

8. Synchronized Bank Account

```

class Account {
    double balance = 5000;

    synchronized void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }

    synchronized void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Insufficient balance");
        }
    }
}

class Deposit extends Thread {
    Account account;

    Deposit(Account account) {
        this.account = account;
    }
}

```

```

        public void run() {
            account.deposit(2000);
        }
    }

    class Withdraw extends Thread {
        Account account;

        Withdraw(Account account) {
            this.account = account;
        }

        public void run() {
            account.withdraw(1000);
        }
    }

    public class SynchronizedBank {
        public static void main(String[] args) {
            Account account = new Account();

            Deposit d = new Deposit(account);
            Withdraw w = new Withdraw(account);

            d.start();
            w.start();
        }
    }

```

9. Fibonacci Thread and Square Thread

```

class NumberStore {
    int number;
    boolean available = false;

    synchronized void put(int n) {
        number = n;
        available = true;
        notify();
    }

    synchronized int get() {
        while (!available) {
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }

        int n = number;
        available = false;
        notify();
        return n;
    }
}

```

```

}

class FibonacciThread extends Thread {
    NumberStore store;

    FibonacciThread(NumberStore store) {
        this.store = store;
    }

    public void run() {
        int a = 0;
        int b = 1;

        for (int i = 0; i <= 20; i++) {
            store.put(a);

            int c = a + b;
            a = b;
            b = c;
        }

        store.put(-1);
    }
}

class SquareThread extends Thread {
    NumberStore store;

    SquareThread(NumberStore store) {
        this.store = store;
    }

    public void run() {
        while (true) {
            int n = store.get();

            if (n == -1)
                break;

            System.out.println("Number: " + n + " Square: " + (n * n));
        }
    }
}

public class FibonacciSquare {
    public static void main(String[] args) {
        NumberStore store = new NumberStore();

        FibonacciThread f = new FibonacciThread(store);
        SquareThread s = new SquareThread(store);

        f.start();
        s.start();
    }
}

```